DATABASE MANAGEMENT SYSTEM

Database Creation & Table Implementation

Creating the online_shop database and its tables with SQL DDL

Submitted By	
Name	Palle Sai Rithesh
Roll Number	2511021240013
Course	BCA Game Development
Subject	Database Management System
Assignment	Assignment 5 — Database Creation & Table Implementation
Project	Online Shopping System

Assignment 5 — requirements

- Create the database for your proposed system.
- Implement all the required tables using SQL DDL commands, based on your finalized database design.
- Ensure that the tables and their relationships are properly defined.

01 TASK 1

Finalized Design Used for Implementation

The tables below come from the Assignment 2 schema, with the improvements proposed in Assignment 3 applied.

The Online Shopping System is implemented as **ten tables** in a MySQL 8 database named **online_shop**. Eight tables are carried over from Assignment 2. Two are added by the normalisation work in Assignment 3: **addresses** (a customer can save several delivery addresses) and **pincodes** (city and state depend on the pincode, so they are stored once).

Table	Purpose	Primary key	Foreign keys
customers	Registered shoppers and their contact details	customer_id	—
pincodes	Pincode to city and state lookup	pincode	—
addresses	Saved delivery addresses of a customer	address_id	customer_id, pincode
categories	Product categories such as Electronics or Books	category_id	—
products	Items on sale with price and stock	product_id	category_id
orders	One row per order placed by a customer	order_id	customer_id, address_id
order_items	Products inside an order (resolves the M:N link)	order_item_id	order_id, product_id
payments	Payment attempts made against an order	payment_id	order_id
shipments	Courier shipments that fulfil an order	shipment_id	order_id
reviews	Ratings and comments written by customers	review_id	customer_id, product_id

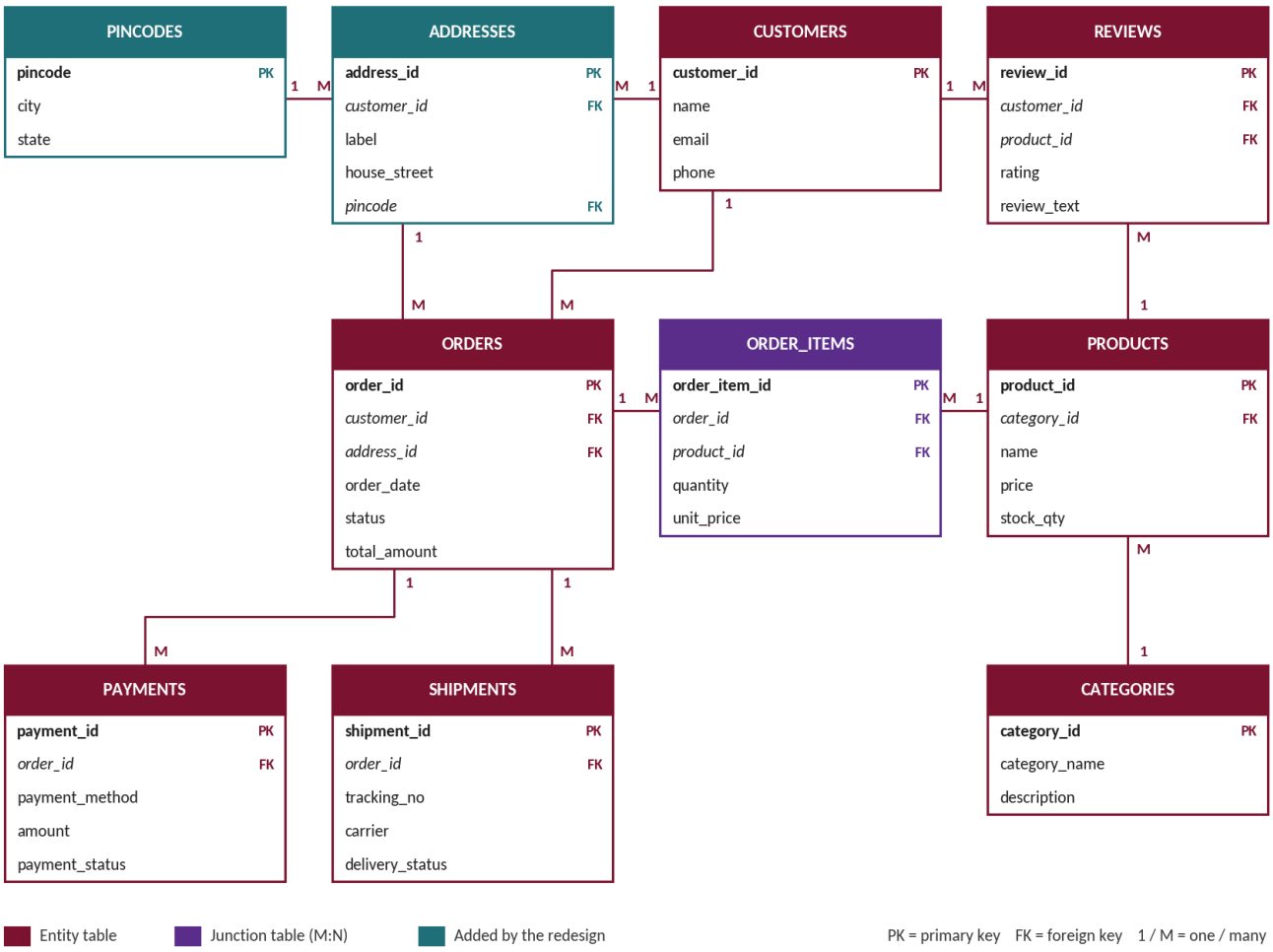


Figure 1 — Relational schema of the implemented database (primary and foreign keys shown).

02 TASK 2

Creating the Database

A database is a named container. All ten tables live inside it.

```
CREATE DATABASE online_shop;
USE online_shop;
```

CREATE DATABASE makes the empty container and **USE** selects it, so every statement that follows runs inside *online_shop*. Confirming that it exists:

```
SHOW DATABASES LIKE 'online_shop';
```

OUTPUT

Database (online_shop)
online_shop

(1 row)

03 TASK 3

Table Implementation with SQL DDL

Tables are created parents first, so every foreign key points at a table that already exists.

The order is: **customers, pincodes, addresses, categories, products, orders, order_items, payments, shipments, reviews**. Every table has a single-column primary key, `NOT NULL` on mandatory columns, and named foreign keys. The `InnoDB` engine is used because it is the MySQL storage engine that enforces foreign keys.

```
CREATE TABLE customers (
    customer_id  INT AUTO_INCREMENT,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(120) NOT NULL,
    phone        VARCHAR(15),
    CONSTRAINT pk_customers PRIMARY KEY (customer_id)
) ENGINE=InnoDB;

CREATE TABLE pincodes (
    pincode  CHAR(6)      NOT NULL,
    city     VARCHAR(60)  NOT NULL,
    state    VARCHAR(60)  NOT NULL,
    CONSTRAINT pk_pincodes PRIMARY KEY (pincode)
) ENGINE=InnoDB;

CREATE TABLE addresses (
    address_id    INT AUTO_INCREMENT,
    customer_id   INT          NOT NULL,
    label         VARCHAR(20)  NOT NULL,
    house_street  VARCHAR(150) NOT NULL,
    pincode       CHAR(6)      NOT NULL,
    CONSTRAINT pk_addresses PRIMARY KEY (address_id),
    CONSTRAINT fk_addresses_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id) ON DELETE CASCADE,
    CONSTRAINT fk_addresses_pincode FOREIGN KEY (pincode)
        REFERENCES pincodes (pincode)
) ENGINE=InnoDB;

CREATE TABLE categories (
    category_id    INT AUTO_INCREMENT,
    category_name  VARCHAR(80)  NOT NULL,
    description    VARCHAR(255),
    CONSTRAINT pk_categories PRIMARY KEY (category_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE products (
    product_id   INT AUTO_INCREMENT,
    category_id  INT           NOT NULL,
    name         VARCHAR(150)  NOT NULL,
    price        DECIMAL(10,2) NOT NULL,
    stock_qty    INT           NOT NULL,
    CONSTRAINT pk_products PRIMARY KEY (product_id),
    CONSTRAINT fk_products_category FOREIGN KEY (category_id)
        REFERENCES categories (category_id)
) ENGINE=InnoDB;

CREATE TABLE orders (
    order_id      INT AUTO_INCREMENT,
    customer_id   INT           NOT NULL,
    address_id    INT           NOT NULL,
    order_date    DATETIME      NOT NULL,
    status        VARCHAR(20)   NOT NULL,
    total_amount  DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_orders PRIMARY KEY (order_id),
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id),
    CONSTRAINT fk_orders_address FOREIGN KEY (address_id)
        REFERENCES addresses (address_id)
) ENGINE=InnoDB;

CREATE TABLE order_items (
    order_item_id  INT AUTO_INCREMENT,
    order_id       INT           NOT NULL,
    product_id     INT           NOT NULL,
    quantity       INT           NOT NULL,
    unit_price     DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_order_items PRIMARY KEY (order_item_id),
    CONSTRAINT fk_order_items_order FOREIGN KEY (order_id)
        REFERENCES orders (order_id) ON DELETE CASCADE,
    CONSTRAINT fk_order_items_product FOREIGN KEY (product_id)
        REFERENCES products (product_id)
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id      INT AUTO_INCREMENT,
    order_id        INT           NOT NULL,
    payment_method  VARCHAR(30)   NOT NULL,
    amount          DECIMAL(10,2) NOT NULL,
    payment_status  VARCHAR(30)   NOT NULL,
    CONSTRAINT pk_payments PRIMARY KEY (payment_id),

    CONSTRAINT fk_payments_order FOREIGN KEY (order_id)
        REFERENCES orders (order_id)
) ENGINE=InnoDB;

CREATE TABLE shipments (
    shipment_id      INT AUTO_INCREMENT,
    order_id         INT          NOT NULL,
    tracking_no      VARCHAR(50)  NOT NULL,
    carrier          VARCHAR(60)  NOT NULL,
    delivery_status  VARCHAR(30)  NOT NULL,
    CONSTRAINT pk_shipments PRIMARY KEY (shipment_id),
    CONSTRAINT fk_shipments_order FOREIGN KEY (order_id)
        REFERENCES orders (order_id)
) ENGINE=InnoDB;

CREATE TABLE reviews (
    review_id    INT AUTO_INCREMENT,
    customer_id  INT           NOT NULL,
    product_id   INT           NOT NULL,
    rating       INT           NOT NULL,
    review_text  VARCHAR(500),
    CONSTRAINT pk_reviews PRIMARY KEY (review_id),
    CONSTRAINT fk_reviews_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id) ON DELETE CASCADE,
    CONSTRAINT fk_reviews_product FOREIGN KEY (product_id)
        REFERENCES products (product_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

> **Note: UNIQUE, CHECK and DEFAULT rules are added in Assignment 6** with ALTER TABLE, so this assignment stays focused on structure: tables, columns, data types, primary keys and foreign keys.

04 TASK 4

Verifying the Tables and Relationships

Standard MySQL commands confirm that everything was created as designed.

4.1 List the tables

```
SHOW TABLES;
```

OUTPUT

Tables_in_online_shop
addresses
categories
customers
order_items
orders
payments
pincodes
products
reviews
shipments

(10 rows)

4.2 Inspect a table's structure

```
DESCRIBE orders;
```

OUTPUT

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	auto_increment
customer_id	int	NO	MUL	NULL	
address_id	int	NO	MUL	NULL	
order_date	datetime	NO		NULL	
status	varchar(20)	NO		NULL	
total_amount	decimal(10,2)	NO		NULL	

(6 rows)

```
DESCRIBE order_items;
```

OUTPUT

Field	Type	Null	Key	Default	Extra
order_item_id	int	NO	PRI	NULL	auto_increment
order_id	int	NO	MUL	NULL	
product_id	int	NO	MUL	NULL	
quantity	int	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	

(5 rows)

PRI marks the primary key. **MUL** marks a column that is indexed but not unique, which is what MySQL creates automatically for each foreign key column.

4.3 List every foreign key relationship

```
SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'online_shop' AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
```

OUTPUT

TABLE_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
addresses	customer_id	customers	customer_id
addresses	pincode	pincodes	pincode
order_items	order_id	orders	order_id
order_items	product_id	products	product_id
orders	address_id	addresses	address_id
orders	customer_id	customers	customer_id
payments	order_id	orders	order_id
products	category_id	categories	category_id
reviews	customer_id	customers	customer_id
reviews	product_id	products	product_id
shipments	order_id	orders	order_id

(11 rows)

4.4 How each relationship is defined

Relationship	Cardinality	Implemented as
Customer places Order	1 : M	orders.customer_id references customers
Customer saves Address	1 : M	addresses.customer_id references customers (cascade on delete)
Pincode locates Address	1 : M	addresses.pincode references pincodes
Address receives Order	1 : M	orders.address_id references addresses
Category groups Product	1 : M	products.category_id references categories
Order contains Product	M : N	order_items table holds order_id and product_id
Order paid by Payment	1 : M	payments.order_id references orders
Order shipped as Shipment	1 : M	shipments.order_id references orders
Customer writes Review	1 : M	reviews.customer_id references customers (cascade on delete)
Product receives Review	1 : M	reviews.product_id references products (cascade on delete)

Note: Conclusion. Ten tables and eleven foreign keys now implement the finalized design. Line items are removed together with their order (ON DELETE CASCADE), while payments and shipments keep the default restrictive behaviour so financial and delivery records cannot disappear by accident.